The cutoff-indexed expansion of the two-dimensional block

Claude, with Daniel Murfet

2026-09-07

Abstract

Assembly of units 104–106. For the equal-exponent block $(h_1+1)/k_1 = (h_2+1)/k_2 = p$ with a general amplitude $\Phi(u,v,s)$ admitting compatible face jets $a_i(v,s)$, $b_j(u,s)$ and corner jets $c_{ij}(s)$ (truncation lengths M_1, M_2 with $(M_1-1)/k_1 < M_2/k_2$ and $(M_2-1)/k_2 < M_1/k_1$) and a mixed remainder $|R| \le C u^{M_1} v^{M_2} (1+s)^D e^{\beta s L}$,

$$Z_\Phi(N) = \sum_{i<M_1} S_{a_i}(N) + \sum_{j<M_2} S_{b_j}(N) - \sum_{i<M_1} \sum_{j<M_2} S_{c_{ij}}(N) \\ + O\big(N^{-(p+\min(M_1/k_1, M_2/k_2))}(1+\log N)\big),$$

where each S is the explicit finite sum of transferred terms of a face expansion: powers $N^{-(p+m/k)}$ with a logarithm exactly at the collisions $i/k_1 = m/k_2$. This is the τ-cutoff theorem of Astra #4(b) with $\tau = \min(M_1/k_1, M_2/k_2)$, in the form "all terms listed, some below the remainder order". Zero `sorry/axiom`.

1 The things formalised

```
structure FaceMoments (  p  b :  ) (h  k  k  M :  ) (Ψ :   →   →  ) (f :   →   →  )
    (H :   →  ) : Prop   -- the three moment hypotheses of face_transfer_bound
noncomputable def faceSum (  p  b :  ) (h  k  k  M :  ) (Ψ :   →   →  ) (f :   →   →  )
    (N :  ) :   :=
   i, transferTerm  (face  p  ((k  :  ) * p  - h  - 1) k  M i) (faceJ M i)
    (faceC ((k  :  ) * p  - h  - 1) b k  k  Ψ f M i) N
theorem face_isBigO ... :
    (fun N :   => twoDAmp   b N h  h  k  k  (fun u _ s => Ψ u s)
        - faceSum   p  b h  k  k  M Ψ f N)
      =O[atTop] fun N :   =>
        N ^ (-(p  + ((M  :  ) - ((k  :  ) * p  - h  - 1)) / k )) * (1 + Real.log N)
theorem isBigO_rpow_log_mono (a a' :  ) (h : a'   a) :
    (fun N :   => N ^ (-a) * (1 + Real.log N)) =O[atTop]
      fun N :   => N ^ (-a') * (1 + Real.log N)
def cornerData (c :   →  ) (m :  ) (s :  ) :   := if m = 0 then c s else 0
theorem twoD_cutoff_isBigO (  b p :  ) (h  h  k  k  M  M  :  ) (h  : 0 <  ) (hb : 0 < b)
    (hk  : 0 < k ) (hk  : 0 < k ) (hp  : ((h  :  ) + 1) / k  = p)
    (hp  : ((h  :  ) + 1) / k  = p)
    (hM  : ((M  :  ) - 1) / k  < (M  :  ) / k ) (hM  : ((M  :  ) - 1) / k  < (M  :  ) / k )
    (Φ :   →   →   →  ) (a bj :   →   →   →  ) (c :   →   →   →  )
    (continuity of Φ, a i, bj j, c i j)
    (Ha Hb :   →   →  ) (measurable, nonnegative)
    (hremA :   i, i < M  →   s, 0 < s →   v   Ioc (0 :  ) b,
      |a i v s -   m   Finset.range M , c i m s * v ^ m|   Ha i s * v ^ M )
```

```
(hremB :   j, j < M  →   s, 0 < s →   u   Ioc (0 :  ) b,
  |bj j u s -   m   Finset.range M , c m j s * u ^ m|   Hb j s * u ^ M )
(C L :  ) (D :  ) (hC : 0   C)
(hmix :   u   Icc (0 :  ) b,   v   Icc (0 :  ) b,   s, 0   s →
  |rectRem Φ a bj c M  M  u v s|
     C * (u ^ M  * v ^ M ) * ((1 + s) ^ D * Real.exp (  * s * L)))
(hmomA hmomB hmomC : FaceMoments for the u-faces, v-faces and corners) :
(fun N :   => twoDAmp   b N h  h  k  k  Φ
    - ((  i   Finset.range M ,
          faceSum   ((((h  + i :  ) :  ) + 1) / k ) b h  k  k  M  (a i)
            (fun m => c i m) N)
      + (  j   Finset.range M ,
          faceSum   ((((h  + j :  ) :  ) + 1) / k ) b h  k  k  M  (bj j)
            (fun m => c m j) N)
      -   i   Finset.range M ,   j   Finset.range M ,
          faceSum   ((((h  + j :  ) :  ) + 1) / k ) b (h  + i) k  k  (M  + k  * M )
            (fun _ s => c i j s) (cornerData (c i j)) N))
  =O[atTop] fun N :   =>
    N ^ (-(p + min ((M  :  ) / k ) ((M  :  ) / k ))) * (1 + Real.log N)
```

2 Mathematics

By twoDAmp_rect the block integral is the sum of the face, corner and remainder pieces. A u-face $u^i a_i(v,s)$ becomes, after absorbing the weight and swapping the coordinates, the one-variable amplitude a_i with parameters (h_2, h_1+i, k_2, k_1); its face expansion (unit 104) has $p_2 = p + i/k_1$, $\gamma = k_2 i/k_1$, remainder exponent $p_2 + (M_2-\gamma)/k_2 = p + M_2/k_2$, and the condition $\gamma < M_2$ is exactly $(M_1-1)/k_1 < M_2/k_2$ (integrality of $i < M_1$ is used). A v-face is symmetric with remainder exponent $p + M_1/k_1$. A corner monomial $c_{ij}(s)u^i v^j$ is the constant amplitude with exponents (h_1+i, h_2+j), i.e. a face term with Taylor data $(c_{ij}, 0, 0, \dots)$ and zero remainder, truncation length $M_1 + k_1 M_2 > \gamma = k_1 j/k_2 - i$, remainder exponent $p + (M_1 + k_1 M_2 + i)/k_1 \geq p + M_1/k_1$. The mixed remainder is $O(N^{-\min(p+M_1/k_1, p+M_2/k_2)}(1+\log N))$ by unit 105. All remainders are at least as small as $N^{-(p+\tau)}(1+\log N)$, $\tau = \min(M_1/k_1, M_2/k_2)$ (isBigO_rpow_log_mono), and IsBigO.sum/add/sub assemble.

3 Paper-facing meaning

This is the higher-order $d = 2$ theorem promised in Astra #3(b) and #4(b): all terms $N^{-(p+q)}(A_q \log N + B_q)$ with q in the two lattices $\mathbb{N}/k_1 \cup \mathbb{N}/k_2$ below the cutoff appear, with $A_q \neq 0$ only at collisions $i/k_1 = m/k_2$ (the resonant faceLogCoeff), the nonlogarithmic coefficients contain regularised axis integrals (faceFPCoeff, the finite parts of unit 100), and the remainder is $O(N^{-(p+\tau)}(1+\log N))$. Terms whose exponent exceeds $p+\tau$ are listed but could be dropped; grouping the coefficients by exponent (the collision bookkeeping Λ_τ) and the explicit smooth specialisation (jets from derivatives, moment hypotheses from envelopes) are the natural follow-ups. Combined with the strict-gap parameter wrapper (unit 99) this reaches the mixed critical/noncritical blocks of thm:TaylorTree.

4 Lean notes

- Bundle repeated integrability hypotheses in a `Prop` structure (`FaceMoments`); the assembly theorem then takes three quantified instances instead of nine.

- `IsBigO.sum` infers the summand family `A` by higher-order unification, which fails when the summand is a bare lambda inside `congr_left`; introduce the families with `set FA :   →   →   := fun i N => …` and state the pieces as `FA i =O g`; the assembled function is then written with Pi-sums applied to `N`, matching the output shape of `IsBigO.add/sub` exactly (`Finset.sum_apply` in the identity proof).

- Over ℝ, `i < M` does not give `i   M - 1`; take `Finset.mem_range.1 hi : i + 1   M` (definitionally) and cast.

- `push_cast; field_simp` typically leaves a commutativity goal; finish with `ring`.